

ML-based Anti-Cheat Systems: Datasets, Data Collection, and Model Architectures for FPS Games

Overview

Machine-learning anti-cheat systems for FPS games rely on large volumes of labeled gameplay telemetry (inputs, movement, aim, actions) to statistically distinguish human play from machine-assisted or exploit-based behavior. Recent research and industry reports emphasize that the main bottleneck is not model capacity but access to high-quality, real-world datasets that capture multiple cheat types and levels of sophistication. For a bachelor thesis project, combining public datasets with data captured from controlled private servers is both feasible and aligned with state-of-the-art work.^{[1][2][^3]}

Public Datasets Relevant to Game Cheating

Cheating in Multiplayer Online Games Dataset (Arxiv 2606.06013)

A recent paper titled "Cheating in Multiplayer Online Games: a Dataset" introduces a dataset specifically focused on cheating behavior in online games. The authors provide structured telemetry with labels indicating honest vs cheating sessions and discuss multiple cheat categories, making it directly relevant as a starting point for anti-cheat modeling. Although details vary by game and release format, the dataset demonstrates that curated telemetry with explicit cheating labels is now available in the literature, which you can reference as a benchmark.^{[4][2]}

Hawk Framework Dataset for CS:GO

The HAWK framework ("Identify As A Human Does: A Pathfinder of Next-Generation Anti-Cheat Framework for First-Person Shooter Games") proposes a server-side anti-cheat system for CS:GO and evaluates it on "the first large and real-world datasets containing multiple cheat types and cheating sophistication". While the full dataset may not be public, the paper describes multi-view features (aim-related, movement-related, timing-based) and label acquisition via expert annotations and official bans, which you can mirror when designing your own private-server experiment.^{[3][5]}

CSGO Cheating Dataset (Kaggle)

A Kaggle dataset called "CSGO cheating dataset" contains engagements between Counter-Strike players, with 10,000 clean players and 2,000 cheaters stored as NumPy arrays. This dataset is designed for supervised learning and is already used in educational and research projects to train models that classify players or engagements as cheating/non-cheating. For a thesis, it can serve as a baseline dataset for experimenting with classical ML and deep learning methods before you integrate your own collected data.^{[6][7]}

Vision-based Cheat Detection Datasets

Jonnalagadda et al. propose a robust vision-based cheat detection approach that operates on final frame buffer images to detect illicit overlays, and they train and evaluate a DNN detector on a dataset collected from two FPS games and three cheating software. VADNet introduces a visual-based anti-cheat CNN architecture for FPS games using raw game video frames, highlighting that frame-level image datasets can also be used to detect anomalies such as aimbots. These works are useful mainly if you want to explore computer vision architectures or augment telemetry-based approaches with visual evidence.^{[8][9][10]}

Esports Trajectories and Actions (ESTA)

The ESTA dataset provides 8.6 million player actions, 7.9 million game frames, and 417,000 player trajectories from around 1,500 professional CS:GO matches. ESTA is not a cheating dataset per se; instead, it is a high-quality dataset for win probability prediction and performance modeling, with benchmarks using gradient-boosted trees and set-based deep learning models (DeepSets, Set Transformer). For anti-cheat, ESTA is valuable as a source of high-level structure for modeling player trajectories and interactions, and as an example of how to represent FPS game states as sets of entities and actions.^{[11][12][13]}

Industry and Academic Frameworks for Data Collection

Frameworks that Safely Integrate Cheats

A KAUST paper proposes a framework that lets developers integrate and test cheats within multiplayer games to safely evaluate cheating behaviors. The framework automates the execution of network flow disruption cheats and logs gameplay data to study synchronization effects, thereby generating labeled data under controlled conditions. This validates your idea of buying or obtaining cheats and running them on private servers: the literature explicitly describes similar setups as a legitimate way to collect structured cheat data.^{[14][15]}

Addressing packet-based cheats, another work focuses on network-level anomalies and formalizes how cheats affect packet timing and consistency, offering a data schema for network packets that can be labeled as normal or manipulated. If your thesis focuses on network/telemetry rather than client internals, these frameworks provide a blueprint for safe, reproducible cheat experiments.^[16]

Server-side Behavioral Telemetry Pipelines

Industry analyses of "AI-native anti-cheat infrastructure" argue that the winning approach is server-side behavioral analysis built on models trained on full telemetry of human play: input timing distributions, micro-adjustment patterns, reaction latency curves, aim trajectories, and decision sequencing. The recommended architecture ingests enormous volumes of event streams (e.g., Kafka), builds player profiles in feature stores, and feeds online detection models that compute a cheating score per session with auditable evidence.^{[17][18][19]}

In practice, this means instrumenting your private server to log every relevant event: player position, view angles, weapon fire events, hit/miss outcomes, reaction time to stimuli, and

sequence patterns over time. Even at small scale, the same telemetry schema can be used to train and evaluate models, then extended later if you work with public or larger datasets.[¹⁸]

Community-based and Telemetry-focused Studies

Earlier work on "Machine Learning to identify cheaters in online games" and community-based investigations of cheating highlight the use of features such as kill/death ratios, headshot rates, accuracy per weapon, and session-level aggregates to detect anomalies. Shaikh et al. explore "new usage of telemetry for anti-cheating in FPS game", focusing on how server-side logs can be mined for suspicious patterns. CleanCombat CS:GO, an open-source project, fetches and stores user stats via the Steam Web API and applies anomaly detection techniques (median absolute deviation) on normalized weapon accuracy ratios to estimate cheating probability.[²⁰][²¹][²²][²³]

These works reinforce that even relatively simple aggregate statistics can be useful, especially when combined with robust estimators and anomaly detection rather than only high-capacity neural networks.[²¹]

Architectures Used in ML-based Anti-Cheat

Classical Machine Learning and Anomaly Detection

Several projects use classical ML methods such as gradient-boosted trees, random forests, logistic regression, and anomaly detection techniques (e.g., MAD-based z-scores) on engineered features. For example, CleanCombat computes normalized gun accuracy features per player, then uses median absolute deviation to find statistical anomalies that strongly deviate from the median distribution in the database. Kaggle projects built on the CS:GO cheating dataset also implement supervised classification models, comparing performance across algorithms.[⁷][²⁰][²¹][⁶]

For a bachelor thesis, these methods are attractive because they are interpretable and relatively easy to implement, allowing you to focus on feature engineering and evaluation methodology without requiring massive deep-learning infrastructure.

Sequence Models for Input and Telemetry Streams

Recent work such as AntiCheatPT (a transformer-based model for Counter-Strike 2) highlights the use of transformer architectures to learn temporal patterns in player telemetry for cheating detection. AntiCheatPT_256 ingests sequences of gameplay events and uses self-attention to capture long-range dependencies in aiming behavior and reaction sequences, improving detection of subtle trigger bots and aimbots over simpler models.[¹]

Industry anti-cheat ML pipelines also suggest sequence models (e.g., LSTMs, GRUs, transformers) for event streams, where each time step encodes position, view angle, target distance, and input events. Such models can be trained on labeled sessions to estimate a cheating probability based on temporal dynamics rather than only static aggregates.[¹⁸]

Set-based Models for Team and Multi-entity States

The ESTA benchmarks show that set-learning models such as DeepSets and Set Transformers can effectively process unordered sets of entities (e.g., multiple players and their features at a given frame) for tasks like win probability prediction. The same idea can be applied to anti-cheat tasks: representing each frame as a set of player feature vectors and using a set transformer to detect abnormal behavior relative to team state or opponent configuration.^[11]

For a bachelor-level project, you could combine a simple per-player sequence model with set-based context features (e.g., average accuracy of teammates and opponents) to reduce false positives by distinguishing truly exceptional performance from suspected cheating.

Vision-based CNN Architectures

Vision-based cheat detection approaches use convolutional neural networks (CNNs) or specialized architectures like VADNet to analyze rendered frames or overlays. Jonnalagadda et al. train DNNs on images with and without cheat overlays, using interval bound propagation for robustness, and report that reliable overlay detection is feasible. VADNet operates directly on FPS game video frames and uses customized modules to spot anomalous aim trajectories and visual cues.^{[9][10]}

These architectures are suitable if your anti-cheat focus is on detecting external overlays or visual anomalies, but they require video capture, dataset construction, and more compute, which may be heavy for a thesis project unless you narrow the scope.

Data Types and Feature Design

Input-level Features

Input-level features include raw mouse movements, click timings, keyboard events, and their temporal relationships. AI-native anti-cheat proposals emphasize reaction latency curves, micro-adjustment patterns in aim, and jitter characteristics as key signals distinguishing human and machine-assisted play. In practice, you might log:^{[19][1][18]}

- Time series of view angle deltas per frame.
- Time between target appearance and first shot.
- Distribution of overshoot/undershoot relative to target center.

These can be aggregated per engagement or processed as sequences by models.

Aim and Accuracy Features

Traditional telemetry-based anti-cheat features include per-weapon accuracy, headshot percentage, damage per shot, and burst patterns. CleanCombat normalizes per-weapon accuracies by overall accuracy and uses robust anomaly detection to find players whose distribution of per-weapon performance is statistically implausible. Combining such aggregate features with sequence-level signals offers a strong baseline.^{[20][21]}

Movement and Positional Features

Movement features capture strafe patterns, positional entropy, and path choices relative to map objectives. Hawk's multi-view features include movement-related views alongside aim features, and ESTA's trajectories show how professional players move and take positions. Including movement features helps distinguish legit aim training from suspicious behavior (e.g., always being in perfect pre-aim position with unnatural consistency).^{[3][11]}

Network and Packet Features

Frameworks addressing network packet-based cheats focus on packet timing, loss, ordering, and integrity. Logging network packets with metadata (timestamps, sequence numbers, sizes) allows detection of speed hacks, packet injection, or desynchronization patterns characteristic of cheating. For your thesis, you could include simple latency and packet-rate features if your server setup exposes them.^{[14][16]}

Integrating Public Datasets with Private Server Data

Using Public Datasets for Baselines

Public datasets like the CSGO cheating dataset and vision-based cheat datasets can be used to train baseline models and to validate feature importance. Because these datasets are already labeled, you can experiment with architectures (e.g., gradient-boosted trees, simple transformers) and evaluation metrics (precision, recall, ROC curves, false positive rates) without building your own logging infrastructure initially.^{[10][6][^7]}

Once you have a working pipeline on public data, you can port the same feature engineering and model architecture to your private server logs, adjusting only for differences in game version or data schema.^[^3]

Collecting Data via Private Servers and Controlled Cheats

Inspired by frameworks that integrate cheats safely, you can set up a private server, instrument it to log telemetry, and run controlled sessions where volunteers or bots use specific cheat software. Each session is labeled according to whether cheats are active, what type they are (aimbot, trigger bot, wallhack), and their configuration (e.g., FOV, reaction time).^{[15][14]}

From a research ethics standpoint, you should keep these experiments offline, never connect to public ranked matches, and ensure participants are informed and consent to being recorded. The resulting dataset will be small relative to industrial scale but sufficient to demonstrate your architecture and training approach in a thesis project.^[^14]

Data Fusion and Domain Adaptation

You can combine features derived from public datasets and your own telemetry using domain adaptation techniques or simply by training separate models and comparing their behavior. For example, train a classifier on the Kaggle CSGO cheating dataset and evaluate it on your private server data after mapping features (accuracy, headshot rate, etc.) to similar scales.

Alternatively, learn sequence models on your telemetry while using ESTA-style representations to include team context.^{[6][11][^3]}

Even if distributions differ, showing how models trained on one dataset generalize (or fail to generalize) to another demonstrates understanding of dataset bias and the importance of real-world data.

Reference Architecture for Your Thesis Project

High-level System Design

A practical architecture for a bachelor thesis anti-cheat project could follow a simplified version of industry ML pipelines:^{[17][18]}

- **Instrumentation Layer:** Game server plugin or middleware that logs gameplay events (positions, view angles, shots, hits, deaths, round state) and optionally input events.
- **Data Ingestion and Storage:** Events streamed to a backend (e.g., REST API, message queue) and stored in a database (PostgreSQL, MongoDB) or as time-series files (Parquet, NumPy arrays).
- **Feature Extraction Service:** Offline scripts or microservices that aggregate raw events into feature sequences and aggregates per player-session.
- **ML Model Service:** Training environment where you experiment with models (gradient boosting, LSTMs/transformers, set-based models) using public datasets and your own data.
- **Detection Endpoint:** Simple API that, given a session's features, returns a cheating score and an explanation (e.g., which features contributed most), without enforcing bans in production.

This architecture is consistent with both Hawk's server-side framework and AI-native anti-cheat proposals while remaining implementable with standard web dev and ML stacks.^{[19][3]}

Recommended Model Stack for a Bachelor Thesis

Given typical constraints, a reasonable stack might be:

- **Phase 1 (Aggregates + Classical ML):** Use Kaggle's CSGO cheating dataset and any public telemetry data to train gradient-boosted trees or random forests on aggregate features (accuracy, headshot rate, K/D, weapon-specific stats).^{[7][6]}
- **Phase 2 (Sequence Model on Private Telemetry):** Instrument a private server to capture per-engagement sequences (aim deltas, reaction times, shot outcomes) and train a compact transformer or LSTM to classify engagements or sessions as cheating/non-cheating.^{[1][18]}
- **Phase 3 (Context-aware Extension):** Optionally incorporate context features inspired by ESTA and Hawk (team state, movement patterns) and evaluate how they reduce false positives.^{[11][3]}

This staged approach lets you deliver a complete pipeline within thesis scope while showing awareness of more advanced architectures.

Ethical and Practical Considerations

Frameworks that enable cheating detection emphasize the need for controlled environments and clear consent, especially when integrating actual cheat software. You should document that all experiments occur on private servers disconnected from public matchmaking, that no real players are banned or harmed, and that cheat use is strictly for research.^[15]^[14]

Additionally, industry analyses underline the critical importance of minimizing false positives, particularly when distinguishing top human performance from AI-assisted play. In your thesis, you can highlight this by carefully selecting evaluation metrics (e.g., precision at high recall, calibration curves) and discussing limitations of your dataset size and feature coverage.^[19]

How to Position Your Thesis

By grounding your project in published datasets like the CS:GO cheating dataset and recent frameworks such as Hawk, and by designing a server-side telemetry collection system for controlled cheat experiments, you can present your anti-cheat thesis as a modern, ML-based system aligned with industry trends. Structuring the work around:^[6]^[1]^[^3]

- Survey of existing datasets and architectures.
- Design of a server-side telemetry and labeling framework.
- Implementation of at least one classical ML baseline and one sequence-based model.
- Evaluation using both public and self-collected data.

will demonstrate originality, feasibility, and technical depth appropriate for a bachelor thesis in web development and system design.^[17]^[19]

References

1. [AntiCheatPT: A Transformer-Based Approach to Cheat ...](#) - This paper presents AntiCheatPT_256, a transformer-based machine learning model designed to detect c...
2. [Cheating in Multiplayer Online Games: a Dataset](#)
3. [A Pathfinder of Next-Generation Anti-Cheat Framework for First ...](#) - The gaming industry has experienced substantial growth, but cheating in online games poses a signifi...
4. [Cheating in Multiplayer Online Games: a Dataset](#)
5. [A Pathfinder of Next-Generation Anti-Cheat Framework for ...](#) - Hawk, a server-side FPS anti-cheat framework for the popular game CS:GO. Hawk utilizes machine learn...
6. [CSGO cheating dataset](#) - Data includes engagements between counter strike players. Data includes 10 000 clean players and 2 0...
7. [wtyngh/Cheating_Discriminator: Implemented a series of ...](#) - In this project, we implemented a series of machine learning techniques to determine whether a playe...
8. [VADNet: Visual-Based Anti-Cheating Detection Network in ...](#) - This work shows the usage of artificial neural networks (ANN) applied in a massive multiplayer onlin...

9. [Detecting aimbot Cheats in FPS games using Computer Vision](#) - VADNet is a visual-based anti-cheat CNN designed for FPS games. It operates on raw game video frame...
10. [Robust Vision-Based Cheat Detection in Competitive Gaming](#) - by A Jonnalagadda · 2021 · Cited by 51 — To this aim, we train and evaluate a DNN detector on a new ...
11. [GitHub - pnxenopoulos/esta: Esports Trajectories and Actions \(ESTA\) dataset](#) - Esports Trajectories and Actions (ESTA) dataset. Contribute to pnxenopoulos/esta development by crea...
12. ["ESTA: An Esports Trajectory and Action Dataset."](#) - Bibliographic details on ESTA: An Esports Trajectory and Action Dataset.
13. [pnxenopoulos - Overview](#) - Research scientist at NVIDIA. pnxenopoulos has 32 repositories available. Follow their code on GitHub...
14. [A Framework to Enable Cheating Detection in Multiplayer ...](#) - by S Shaikh · 2025 · Cited by 4 — in this paper, we propose a framework for developers to integrate ...
15. [B4 Shaikh, S., Bertin, H., Dacier, M., Bromberg, YD.](#) - This conference paper presents a framework designed to assist game developers in detecting cheating ...
16. [Addressing Network Packet-based Cheats in Multiplayer Games](#)
17. [Real-Time Anomaly Detection in 10M+ Daily Game Sessions](#) - A real-time anti-cheat reference architecture: Kafka event streaming for 10M+ daily sessions, Flink ...
18. [Anti-Cheat ML Pipelines | Pixels - Envisioning.io](#) - Server-side machine learning spots aimbots, automation, and exploits by analyzing player telemetry a...
19. [AI-native Anti Cheat Infrastructure - #LeadEarly - Early Game Ventures](#) - Cheating has always been the original sin of competitive gaming—but for most of gaming's history, it...
20. [\[PDF\] Machine Learning to identify cheaters in online games](#)
21. [GitHub - DawsonReschke/CleanCombat CS-GO: Server responsible for fetching and storing user game stats, while also comparing each user against the whole database to determine the probability that the player is cheating.](#) - Server responsible for fetching and storing user game stats, while also comparing each user against ...
22. [\[PDF\] New usage of telemetry for anti-cheating in FPS game](#)
23. [\[PDF\] A Community-based Investigation of Cheating in Online Multiplayer ...](#)